Лабораторная работа 4.
Модульная организация проекта.
На примере проекта «Расчёт скорости резания при токарной обработке»

Цель работы: Разработать **модульное консольное приложение** для расчёта скорости резания при токарной обработке, в котором:

- применяются условные операторы if-elif-else для проверки границ значений;
- применяется конструкция match-case для маршрутизации логики (выбор режима обработки, название режима);
- организуется циклический ввод данных до получения корректного значения;
- программа **разделена на модули** с чёткой зоной ответственности;
- оформлена точка входа if __name__ == "__main__";
- реализованы **юнит-тесты** (unittest) для модулей validators и operations.

Постановка

Разработать программу, которая:

1. Выводит меню выбора режима обработки.
2. Запрашивает у пользователя режим, диаметр заготовки D и частоту вращения n.
3. Проверяет корректность каждого ввода (тип и диапазон).
4. Рассчитывает скорость резания V и скорректированную скорость V_adj.
5. Классифицирует диаметр заготовки.
6. Выводит результаты: режим, V, V_adj, класс диаметра.
7. Позволяет выполнить несколько расчётов подряд до выхода.

Требуемая структура проекта

```
turning_project/
│
├── main.py           # точка входа, главный цикл программы
├── constants.py      # константы: диапазоны, коэффициенты, пороги
├── validators.py     # валидация ввода (if-elif-else, while, try/except)
├── operations.py     # расчёты V, V_adj, класс диаметра
├── menu.py           # меню и маршрутизация через match-case
├── logger.py         # журнал ошибок ввода
├── tests.py          # unittest-тесты для validators и operations
├── README.md         # описание проекта и инструкция запуска
└── .vscode/
    └── launch.json   # конфигурация запуска в VS Code
```

Состав проекта:

Файл **constants.py** — константы проекта(согласно заданию варианта)

Элемент	Тип	Значение / назначение
K_COARSE	float	1.0 — коэффициент черновой обработки
K_SEMI	float	0.85 — получистовая
K_FINISH	float	0.7 — чистовая
D_MIN, D_MAX	float	1.0, 500.0 — диапазон диаметра

Элемент	Тип	Значение / назначение
N_MIN, N_MAX	float	50.0, 3000.0 — диапазон частоты
D_SMALL_LIMIT	float	10.0 — порог малого диаметра
D_LARGE_LIMIT	float	100.0 — порог крупного диаметра
PI	float	math.pi
LOG_FILE	str	"input_errors.log" — имя файла журнала

Требования:

- docstring модуля;
- импорт math для PI;
- никаких функций — только константы.

Файл logger.py — журнал ошибок ввода, позволяющий фиксировать все некорректные вводы пользователя (для отчёта и анализа).

Требования:

- внутренний список _entries: list[str];
- формат записи: [YYYY-MM-DD HH:MM:SS] сообщение;
- запись в файл — в try/except OSError (если файл недоступен, программа не должна падать);
- docstring у модуля и у каждой функции.

Обязательно должно быть:

Функция	Сигнатура	Назначение
log_error	log_error(message: str) -> None	добавить запись в журнал; дописать в файл LOG_FILE с меткой времени
get_entries	get_entries() -> list[str]	вернуть копию журнала в памяти
clear	clear() -> None	очистить журнал (используется в тестах)

Файл validators.py — валидация ввода, позволяет проверять тип и диапазон вводимых значений, обеспечивать циклический ввод до получения корректного значения.

Требования к методу input_number:

- цикл while not is_valid с флагом — по варианту задания;
- при ValueError — сформировать **строку** сообщения (не *кортеж* e.args), вывести её, записать через log_error, продолжить цикл;
- при выходе за границы — сформировать строку ошибки через validate_range, вывести, записать, продолжить;
- при корректном значении — вернуть float.

Требования к validate_range:

- обязательно if-elif-else (три ветви: ниже, выше, внутри);
- возвращает *кортеж* (True, "") или (False, "сообщение").
  Запрещено: использовать match-case в этом модуле (только if-elif-else и try/except).

Обязательно должно быть:

Функция	Сигнатура	Назначение
validate_range	validate_range(value: float, low: float, high: float) -> tuple[bool, str]	проверка границ **через** if-elif-else: < low, > high, иначе — ok
input_number	input_number(prompt: str, low: float, high: float) -> float	циклический ввод числа: проверка типа через try/except ValueError, проверка границ через validate_range
input_diameter	input_diameter() -> float	обёртка: input_number("Введите диаметр...", D_MIN, D_MAX)
input_frequency	input_frequency() -> float	обёртка: input_number("Введите частоту...", N_MIN, N_MAX)

Файл operations.py — расчёты, должен выполнять арифметические расчёты и классификацию диаметра.
Требования:

- docstring у модуля и у каждой функции;
- описание параметров — в docstring (секция Args:), а **не** в сигнатуре через type.Argument(...);
- **не использовать** from __future__ import annotations (в Python 3.10+ не нужен);
- имя модуля констант — строго constants (не connstans и т.п.);
- если нужны аннотации с описанием — использовать typing.Annotated[float, "описание"].

Обязательно должно быть:

Функция	Сигнатура	Назначение
cutting_speed	cutting_speed(d: float, n: float) -> float	$V = \pi \cdot D \cdot n / 1000$
adjusted_speed	adjusted_speed(v: float, k: float) -> float	$V_{adj} = V \cdot k$
diameter_class	diameter_class(d: float) -> str	классификация через if-elif-else: малый / нормальный / крупный

Файл menu.py — меню и маршрутизация, должен выводить меню и обрабатывать выбор пользователя **через** match-case.
Требования:

- маршрутизация выбора режима — **только** match-case (без цепочки if-elif);
- при неверном пункте меню — сообщение об ошибке и повторный запрос;
- выход из программы — при выборе "0" (функция возвращает None);
- docstring у модуля и у каждой функции.

Обязательно должно быть:

Функция	Сигнатура	Назначение
show_menu	show_menu() -> None	вывести пункты 1–3 и 0 (выход)
select_mode	select_mode() -> float | None	циклический ввод номера режима; match-case по "1", "2", "3", "0", _; возвращает k или None при выходе
mode_name	mode_name(k: float) -> str	match-case по k: 1.0, 0.85, 0.7, _ → название режима

Файл main.py — точка входа
Назначение: связать модули, реализовать главный цикл программы.
Обязательно должно быть:

Элемент	Требование
Импорты	from validators import input_diameter, input_frequency; from menu import show_menu, select_mode, mode_name; from operations import cutting_speed, adjusted_speed, diameter_class
Функция main	def main() -> None: — главный сценарий
Цикл	while True: — вывод меню → выбор режима → если None, выход
Ввод	d = input_diameter(), n = input_frequency()
Расчёты	v = cutting_speed(d, n), v_adj = adjusted_speed(v, k), cls = diameter_class(d)
Вывод	режим + k, V (:.2f), V_adj (:.2f), класс диаметра
Точка входа	if __name__ == "__main__": main()

Требования:
- docstring у модуля и у main;
- from __future__ import annotations **не использовать**;
- никакой логики валидации/расчёта — только связывание модулей;
- запуск: python main.py или **F5** в VS Code.

Файл tests.py — юнит-тесты
Назначение: автоматически проверять корректность validators.py и operations.py.
Обязательно должно быть:

Класс тестов	Что проверяет
TestValidateRange	границы: внутри, на нижней, на верхней, ниже, выше (5 тестов)
TestInputDiameter	циклический ввод: abc → 600 → 50; границы 1, 500 (3 теста)
TestInputFrequency	циклический ввод: 10 → 5000 → 1000; границы 50, 3000 (3 теста)
TestOperations	cutting_speed(100, 1000) ≈ 314.1593; adjusted_speed для трёх k; diameter_class для 5, 10, 50, 100, 150 (9 тестов)

Итого: не менее **20 тестов**.

Требования:

- используется unittest;
- для input_* — подмена input() через unittest.mock.patch("validators.input", side_effect=[...]);
- в setUp — очистка журнала (logger.clear());
- запуск: python -m unittest tests -v → все тесты **OK**;
- docstring у модуля и у каждого класса тестов.

Файл README.md — описание проекта

Обязательно должно содержать:

1. Название проекта и вариант.
2. Требования к среде (Python 3.10+).
3. Инструкцию запуска в VS Code (F5) и в терминале (python main.py).
4. Инструкцию запуска тестов (python -m unittest tests -v).
5. Формулы V и V_adj.
6. Таблицу режимов с коэффициентами k.
7. Допустимые диапазоны D и n.
8. Правила классификации диаметра (<10, 10..100, >100).
9. Краткую таблицу «файл → назначение».

4.9. .vscode/launch.json — конфигурация запуска

Назначение: чтобы **F5** сразу запускал main.py без ручной настройки.

Обязательно должно быть:

- "version": "0.2.0";
- одна конфигурация: "name": "Run main.py", "type": "python", "request": "launch", "program": "${workspaceFolder}/main.py", "console": "integratedTerminal".

Тестовые сценарии для сдачи (обязательны к выполнению)

№	Ввод	Ожидание
1	2, 50, 1000	V ≈ 157.08, V_adj ≈ 133.52, «нормальный диаметр»
2	1, abc, 600, 200, 1500	2 ошибки, затем V ≈ 942.48, «крупный диаметр»
3	3, 5, 3000	V ≈ 47.12, V_adj ≈ 32.99, «малый диаметр»

№	Ввод	Ожидание
4	2, 10, 100	«нормальный диаметр» (граница 10)
5	2, 100, 100	«нормальный диаметр» (граница 100)
6	4 (неверный пункт) → 0	сообщение об ошибке, затем выход
7	0 сразу	«Работа завершена.»
8	python -m unittest tests -v	Ran 20 tests ... OK
9	python -c "import validators"	импорт **без** запуска main